

MrBERT

Dynamic Token Deletion for
Efficient BERT Inference

Technical Documentation

Adapted from MrT5: Dynamic Token Merging for
Efficient Byte-level Language Models

(Kallini et al., 2024)

March 1, 2026

Contents

1	MrBERT Explainer: What We Changed and Why	3
1.1	The Core Idea	3
1.2	Config Changes	3
1.3	The Delete Gate	4
1.3.1	The ScaledSigmoid Activation (Lines 160–169)	4
1.3.2	The SigmoidDeleteGate Module (Lines 178–249)	4
1.4	Soft Deletion: Masking Attention	4
1.4.1	How It Works (Lines 414–419)	5
1.4.2	Softmax1 Variant (Lines 422–425)	5
1.5	Hard Deletion: Physically Removing Tokens	5
1.5.1	Step 1: Decide Which Tokens Survive (Lines 509–541)	5
1.5.2	Step 2: Gather Surviving Tokens (Lines 543–553)	5
1.6	Protecting Special Tokens	6
1.7	Where the Gate Sits in the Encoder	6
1.7.1	Forward Pass Gate Propagation (Lines 670–707)	6
1.8	Training: The Deletion Loss and PI Controller	6
1.8.1	The Deletion Loss (Lines 347–387)	6
1.8.2	The PI Controller (Lines 303–344)	7
1.8.3	The Regularizer Delay (Lines 977–985)	7
1.8.4	Separate Learning Rate for the Gate (Lines 850–866)	7
1.9	Gate Variants	7
1.10	What Stays the Same	7
1.11	End-to-End Flow	8
1.12	Key File Reference	9
2	Training Run Analysis Guide	10
2.1	Concepts: Steps vs Epochs	10
2.2	Launching a Run	10
2.2.1	Recommended Full Training Command (Modal)	10
2.2.2	Key Parameters in DEFAULT_ARGS	11
2.3	W&B Metrics Reference	11
2.3.1	Task Performance Metrics	11
2.3.2	Delete Gate Metrics	11
2.3.3	Optimization Metrics	12
2.4	Step-by-Step Analysis Workflow	12
2.4.1	Phase 1: Steps 0–1,000 (Pre-Regularizer)	12
2.4.2	Phase 2: Steps 1,000–5,000 (PI Controller Engages)	13
2.4.3	Phase 3: Steps 5,000–17,168 (End of Epoch 1)	13
2.4.4	Phase 4: Steps 17,168–51,500 (Epochs 2–3)	13
2.5	Parameter Adjustments by Symptom	13
2.5.1	Deletion Rate Never Reaches Target	13
2.5.2	Gate Not Discriminating (Low delete_gate_std)	13

2.5.3	Task Performance Worse Than BERT Baseline	14
2.5.4	Learning Rate Issues	14
2.6	Success Criteria	14
2.6.1	Task Performance	14
2.6.2	Delete Gate Convergence	14
2.6.3	Training Stability	14
2.6.4	Comparison Against Baseline	15
2.6.5	Summary Table	15
2.6.6	When to Stop Early	15
3	Training Run Plan	16
3.1	Overview	16
3.2	Run 1: BERT Baseline	16
3.3	Run 2: MrBERT Standard	17
3.4	Run 3: MrBERT Higher Deletion Rate	17
3.5	Run 4: MrBERT Lower Deletion Rate	18
3.6	Run 5: MrBERT No PI Controller	18
3.7	Run 6: MrBERT Earlier Gate Layer	19
3.8	Run 7: MrBERT Later Gate Layer	19
3.9	Run 8: MrBERT Random Gate	20
3.10	Summary: What Each Run Tells You	20
3.11	Recommended Run Order	21

Chapter 1

MrBERT Explainer: What We Changed and Why

This chapter explains every modification made to standard BERT to create MrBERT, with the intuition behind each change and pointers to the relevant code.

Reference paper: [MrT5: Dynamic Token Merging for Efficient Byte-level Language Models](#) (Kallini et al., 2024). MrBERT adapts the delete gate mechanism from MrT5 to the BERT architecture.

1.1 The Core Idea

Standard BERT processes every token through all 12 encoder layers, even tokens that carry little semantic information (e.g. punctuation, filler words, repeated content). This is computationally wasteful.

The MrBERT idea: after a chosen encoder layer, assign each token a score. Low-scoring tokens are either ignored by subsequent attention layers (soft deletion) or physically removed from the sequence (hard deletion). The rest of the encoder only processes the surviving tokens.

This is a learned compression: the model is trained to identify and discard tokens that don't contribute to the task, reducing the effective sequence length and therefore the compute required for the remaining layers.

1.2 Config Changes

File: `mrbert/models/configuration_mrbert.py`, lines 48–71

`MrBertConfig` extends the standard `BertConfig` with the following new fields:

Parameter	Default	Purpose
<code>delete_gate_layer</code>	3	Which encoder layer emits the gate (0-indexed). Layers after this one only see surviving tokens.
<code>deletion_type</code>	"scaled_sigmoid"	Which gate variant to use.
<code>sigmoid_mask_scale</code>	-30.0	Controls the strength of the deletion signal added to attention scores.
<code>deletion_threshold</code>	-15.0	Gate value below which a token is counted as deleted (<code>sigmoid_mask_scale / 2</code>).
<code>gate_layer_norm</code>	True	Apply LayerNorm to hidden states before computing the gate.
<code>use_gumbel_noise</code>	False	Add Gumbel noise to gate logits during training to encourage exploration.
<code>random_deletion_probability</code>	0.5	Used only by the random gate ablation.
<code>fixed_deletion_amount</code>	0.5	Used only by the fixed gate ablation.

Table 1.1: New configuration parameters added by `MrBertConfig`.

None of these exist in standard `BertConfig`. They expose the gate as a set of tunable hyperparameters without touching the core BERT architecture.

1.3 The Delete Gate

File: `mrbert/models/modeling_mrbert.py`, lines 160–249

1.3.1 The ScaledSigmoid Activation (Lines 160–169)

```
def forward(self, input):
    return self.sigmoid_mask_scale * torch.sigmoid(-input)
```

This maps the gate’s raw logit to a value in the range $[-30, 0]$:

- **Large positive logit** $\rightarrow \sigma(-\text{large}) \approx 0 \rightarrow \text{gate value} \approx 0 \rightarrow \text{keep the token}$
- **Large negative logit** $\rightarrow \sigma(-\text{large_neg}) \approx 1 \rightarrow \text{gate value} \approx -30 \rightarrow \text{delete the token}$

The output is designed to be added directly to attention scores (see Section 1.4). A value of 0 has no effect; a value of -30 makes the softmax assign essentially zero attention weight to that token.

1.3.2 The SigmoidDeleteGate Module (Lines 178–249)

The gate is a small neural network sitting on top of one encoder layer:

```
hidden_state (768-dim)
|
[LayerNorm]          <- normalizes the input for stable training
|
Linear(768 -> 1)      <- learns which features predict
                        "this token is dispensable"
|
ScaledSigmoid(-30)    <- maps the scalar logit to [-30, 0]
|
gate_value per token
```

Initialization (lines 236–248): The linear layer is initialized with very small weights (`std=0.001`) and a large positive bias (`10.0`). This is critical. With `bias=10`:

$$\text{gate_value} = -30 \times \sigma(-10) \approx -30 \times 0.000045 \approx 0$$

All tokens start with a gate value near 0 (keep everything). The model has to actively learn to push tokens toward -30 . This prevents catastrophic deletion at the start of training before the model has learned anything useful.

1.4 Soft Deletion: Masking Attention

File: `mrbert/models/modeling_mrbert.py`, lines 393–441 (`MrBertSelfAttention.forward`)

Soft deletion is the default mode. Tokens are not removed from memory—they just become invisible to attention in subsequent layers.

1.4.1 How It Works (Lines 414–419)

```
if delete_gate_mask is not None:
    delete_gate_mask_expanded = delete_gate_mask.squeeze(-1) \
                                .unsqueeze(1).unsqueeze(1)
    attention_scores = attention_scores + delete_gate_mask_expanded
```

In standard BERT, attention scores are computed as $QK^T/\sqrt{d}$ and then passed to softmax. In MrBERT, before the softmax, we add the gate values to the attention scores along the key dimension.

Intuition: If token A has gate value -30 , then every other token’s attention score for attending to token A is reduced by 30. After softmax, token A receives essentially zero attention weight from every other token. It is still in the sequence but is completely ignored by all subsequent layers.

This is an elegant approach because:

- It requires no architectural surgery—just an additive bias before softmax.
- It is fully differentiable, so the gate can be trained end-to-end.
- The original BERT attention mechanism is untouched; the gate is purely additive.

1.4.2 Softmax1 Variant (Lines 422–425)

Optionally enabled via `use_softmax1=True`. Adds 1 to the softmax denominator, allowing attention weights to sum to less than 1. This lets the model express “I don’t want to attend to anything strongly” rather than being forced to distribute all attention mass across available tokens.

1.5 Hard Deletion: Physically Removing Tokens

File: `mrbert/models/modeling_mrbert.py`, lines 509–553 (`MrBertLayer`)

Hard deletion is an optional mode (`hard_delete=True`). Instead of masking attention, tokens below the deletion threshold are physically removed from the sequence tensor.

1.5.1 Step 1: Decide Which Tokens Survive (Lines 509–541)

```
keep_this = delete_gate_mask > deletion_threshold # True=keep, False
            =delete
```

A cumulative sum maps each surviving token to its new position in the shorter sequence. A new attention mask is computed for the shortened sequence.

1.5.2 Step 2: Gather Surviving Tokens (Lines 543–553)

```
new_hidden_states = torch.gather(hidden_states, 1, gather_indices)
```

`torch.gather` physically selects only the rows corresponding to kept tokens, reducing the sequence dimension from `seq_len` to `num_kept_tokens`.

Intuition: After hard deletion, layers 4–11 operate on a shorter sequence. Since attention is $O(n^2)$ in sequence length, removing 30% of tokens reduces attention compute in those layers by roughly $0.7^2 \approx 49\%$. This is the actual efficiency gain MrBERT is designed to achieve.

Why soft deletion is the default: Soft deletion is fully differentiable and easier to train. Hard deletion involves discrete decisions (keep/delete) which complicate gradients. In practice, soft deletion is used during training; hard deletion can be applied at inference time.

1.6 Protecting Special Tokens

File: `mrbert/models/modeling_mrbert.py`, lines 213–232 (`SigmoidDeleteGate.forward`)

Special tokens (`[CLS]`, `[SEP]`, `[PAD]`) receive hardcoded gate values:

- `[CLS]` and `[SEP]` are forced to gate value 0 (always kept). `[CLS]` carries the sequence representation used for classification; `[SEP]` marks sentence boundaries.
- `[PAD]` tokens are forced to gate value -30 (always deleted). They carry no information and should never receive attention.

Only non-special tokens pass through the learned gate. This ensures the model cannot break itself by accidentally deleting structurally necessary tokens.

1.7 Where the Gate Sits in the Encoder

File: `mrbert/models/modeling_mrbert.py`, lines 644–647

The delete gate is attached to a single encoder layer, specified by `delete_gate_layer` (default: 3). The gate is computed after layer 3’s standard self-attention and FFN, using the output hidden states as input.

Why layer 3? After 3 layers, the model has built up enough contextual representation for the gate to make meaningful keep/delete decisions. Earlier layers may not have enough context; later layers have already done most of the work and there is less compute to save.

1.7.1 Forward Pass Gate Propagation (Lines 670–707)

During the forward pass, the encoder tracks the `delete_gate_mask` produced at layer 3. This mask is passed as input to every subsequent attention layer (layers 4–11), which apply the soft deletion bias. The gate is computed once and reused for the rest of the encoder.

1.8 Training: The Deletion Loss and PI Controller

File: `mrbert/training/train_mrbert.py`

Training MrBERT requires solving two objectives simultaneously:

1. Minimize the task loss (get good NLI accuracy).
2. Delete approximately the target fraction of tokens.

These two objectives can conflict—the easiest way to minimize task loss is to delete nothing. The deletion loss and PI controller are the mechanism that enforces the deletion target.

1.8.1 The Deletion Loss (Lines 347–387)

```
deletion_loss = delete_gate_output[non_pad_mask].mean()
```

This is simply the mean gate value across all non-pad tokens. Since gate values are in $[-30, 0]$, minimizing this loss pushes gate values toward -30 (more deletion). The task loss pushes the gate toward 0 (keep everything useful). The balance between these two forces determines the actual deletion rate.

1.8.2 The PI Controller (Lines 303–344)

A fixed `deletion_loss_weight` is fragile—too high and the gate collapses (deletes everything), too low and nothing gets deleted. Instead, the weight α is adjusted dynamically by a PI controller:

```
error = target_rate - actual_deletion_rate
p_acc = 0.9 * p_acc + 0.1 * kp * error    # proportional (EMA smoothed)
i_acc = i_acc + ki * error                # integral (accumulates)
alpha = max(0.0, p_acc + i_acc)          # final weight
```

Intuition: If the actual deletion rate is below the target (not deleting enough), `error` is positive, so α increases, applying more deletion pressure. If the rate overshoots, `error` is negative, reducing α . The integral term prevents the rate from perpetually sitting slightly below target—it accumulates the error over time and applies a slow corrective push.

1.8.3 The Regularizer Delay (Lines 977–985)

```
if global_step >= args.regularizer_delay:
    loss = task_loss + current_alpha * deletion_loss
else:
    loss = task_loss
```

For the first `regularizer_delay` steps (default: 1000), only the task loss is optimized. This gives the model time to learn what makes a good NLI prediction before deletion pressure is applied. If deletion pressure starts immediately, the gate may learn to delete tokens before the model knows which ones are important.

1.8.4 Separate Learning Rate for the Gate (Lines 850–866)

```
optimizer = AdamW([
    {"params": other_params, "lr": args.learning_rate},    # 2e-5
    {"params": delete_gate_params, "lr": args.delete_gate_lr}, # 1e-4
])
```

The gate is a randomly initialized small network sitting on top of pretrained BERT weights. It needs to learn faster than the pretrained weights, which only need fine-tuning. A higher learning rate for gate parameters allows the gate to adapt quickly without destabilizing the pretrained representations.

1.9 Gate Variants

File: `mrbert/models/modeling_mrbert.py`, lines 178–346

Four gate implementations are available via `deletion_type` in the config:

The random and fixed gates exist to answer the question: *does learning which tokens to delete actually matter, or is any deletion at the right rate equally good?*

1.10 What Stays the Same

Everything else in the BERT architecture is unchanged:

Type	Class	Lines	Description
scaled_sigmoid	SigmoidDeleteGate	178–249	Main learnable gate. Trained end-to-end.
log_sigmoid	LogSigmoidDeleteGate	251–256	Same architecture, uses log sigmoid instead of scaled sigmoid.
random	RandomDeleteGate	259–299	Ablation: randomly deletes tokens at a configured rate.
fixed	FixedDeleteGate	301–346	Ablation: always deletes the same fixed fraction of non-special tokens.

Table 1.2: Gate variant implementations.

- The tokenizer and vocabulary
- The embedding layer
- The 12-layer transformer encoder structure
- Multi-head self-attention (except the additive mask)
- Feed-forward layers within each encoder layer
- Layer normalization and residual connections
- All task-specific heads (`MrBertForSequenceClassification`, `MrBertForMaskedLM`, etc.)—these are direct ports of the standard BERT task heads

The baseline for comparison is `modeling_bert.py`, which is a standard BERT implementation. Running with `-model-type BERT` uses this file directly, with no gate and no deletion, so results are directly comparable.

1.11 End-to-End Flow

Here is what happens to a single SNLI example (premise + hypothesis) during a MrBERT forward pass:

```

Input tokens: [CLS] The cat sat . [SEP] The animal rested . [SEP] [
  PAD] ...
      |
      BertEmbeddings (unchanged)
      |
      Encoder Layer 0 (standard BERT layer)
      Encoder Layer 1 (standard BERT layer)
      Encoder Layer 2 (standard BERT layer)
      |
      Encoder Layer 3 <- has delete gate
      +-- Standard BERT self-attention + FFN (produces hidden
states)
      +-- SigmoidDeleteGate:
          LayerNorm -> Linear(768->1) -> ScaledSigmoid
          +-----+
          | [CLS]  ->  0.0          (protected)      |
          | The    -> -2.1          (keep)            |
          | cat    -> -0.3          (keep)            |
          | sat    -> -18.4         (delete)           |

```

```

| .      -> -24.1      (delete)      |
| [SEP]  ->  0.0       (protected)   |
| The    -> -19.2      (delete)      |
| animal -> -1.1       (keep)        |
| rested -> -0.8       (keep)        |
| .      -> -22.3      (delete)      |
| [SEP]  ->  0.0       (protected)   |
| [PAD]  -> -30.0      (always deleted) |
+-----+
| delete_gate_mask (passed to layers 4-11)
Encoder Layer 4  <- receives delete_gate_mask
    attention scores += delete_gate_mask
    -> deleted tokens receive ~0 attention weight
Encoder Layer 5  (same)
...
Encoder Layer 11 (same)
|
[CLS] hidden state -> classifier -> entailment / neutral /
contradiction

```

The key insight: layers 4–11 never “see” the deleted tokens in any meaningful sense. The gate has effectively compressed the sequence, and all subsequent computation operates on the surviving tokens only.

1.12 Key File Reference

Component	File	Lines
Config additions	mrbert/models/configuration_mrbert.py	48–71
ScaledSigmoid	mrbert/models/modeling_mrbert.py	160–169
SigmoidDeleteGate	mrbert/models/modeling_mrbert.py	178–249
Special token protection	mrbert/models/modeling_mrbert.py	213–232
Soft deletion	mrbert/models/modeling_mrbert.py	414–419
Hard deletion	mrbert/models/modeling_mrbert.py	509–553
Gate in encoder	mrbert/models/modeling_mrbert.py	644–647
PI controller	mrbert/training/train_mrbert.py	303–344
Deletion loss	mrbert/training/train_mrbert.py	347–387
Regularizer delay	mrbert/training/train_mrbert.py	977–985
Gate learning rate	mrbert/training/train_mrbert.py	850–866
Baseline BERT	mrbert/models/modeling_bert.py	—

Table 1.3: Key source files and line references.

Chapter 2

Training Run Analysis Guide

A step-by-step guide for launching training, interpreting W&B metrics, diagnosing problems, adjusting parameters, and deciding when a run is complete.

2.1 Concepts: Steps vs Epochs

Term	Definition
Step	One forward + backward pass on a single batch of 32 examples. Weights are updated once per step.
Epoch	One full pass through the entire training dataset (~549,367 SNLI examples).

With `batch_size=32` on SNLI:

```
steps_per_epoch ~ 549,367 / 32 ~ 17,168
1 epoch ~ 17,168 steps
3 epochs ~ 51,500 steps
```

Important: the LR scheduler decays linearly from `learning_rate` to 0 over `total_steps`. If `-max-steps` is set lower than the actual number of steps you run, the LR will hit near-zero early and training stalls. Always use `-max-steps -1` for full-epoch runs.

2.2 Launching a Run

2.2.1 Recommended Full Training Command (Modal)

```
# MrBERT with delete gate -- 3 epochs, 30% deletion target
modal run --detach train_modal.py \
  --model-type MrBERT \
  --max-steps -1 \
  --num-epochs 3 \
  --target-deletion-rate 0.3

# BERT baseline -- no delete gate
modal run --detach train_modal.py \
  --model-type BERT \
  --max-steps -1 \
  --num-epochs 3
```

Parameter	Default	Effect
<code>-learning_rate</code>	2e-5	Base LR for BERT weights
<code>-batch_size</code>	32	Examples per step
<code>-target_deletion_rate</code>	0.3	Fraction of non-pad tokens the gate should delete
<code>-deletion_loss_weight</code>	0.01	Initial α_0 for deletion regularizer (PI controller adjusts from here)
<code>-controller_p</code>	0.01	Proportional gain—how fast α responds to deletion rate error
<code>-controller_i</code>	0.00001	Integral gain—slow cumulative correction
<code>-regularizer_delay</code>	1000	Steps before any deletion pressure is applied
<code>-logging_steps</code>	50	How often metrics are logged to W&B
<code>-save_steps</code>	1000	How often checkpoints are saved

Table 2.1: Key training parameters and their defaults.

2.2.2 Key Parameters in `DEFAULT_ARGS`

2.3 W&B Metrics Reference

2.3.1 Task Performance Metrics

Metric	What It Measures	Expected Behavior
<code>loss</code>	Combined loss (task + deletion)	Should decrease steadily, then plateau
<code>cross_entropy_loss</code>	Classification loss only	Primary signal for task learning
<code>accuracy</code>	Training batch accuracy	Should rise from $\sim 33\%$ to $85\%+$
<code>eval/loss</code>	Validation loss	Should track training loss; divergence = overfitting
<code>eval/accuracy</code>	Validation accuracy	The definitive performance metric

Table 2.2: Task performance metrics.

2.3.2 Delete Gate Metrics

Metric	What It Measures	Expected Behavior
<code>percent_non_pad_deleted_tokens</code>	% of non-pad tokens with gate $<$ threshold	Should converge to $\text{target_deletion_rate} \times 100$
<code>percent_deleted_tokens</code>	% of all tokens (incl. pad) deleted	Always lower than the non-pad version

Metric	What It Measures	Expected Behavior
<code>new_seq_len</code>	Avg tokens kept per sequence after deletion	Should stabilize at $\text{avg_seq_len} \times (1 - \text{target_deletion_rate})$
<code>delete_gate_average</code>	Mean gate value across all tokens	Near 0 at init; rises/falls as gate learns
<code>delete_gate_std</code>	Spread of gate values within a sequence	Low std = gate treating all tokens similarly (bad); high std = discriminating
<code>delete_gate_max/min</code>	Range of gate values	Max near 0 (keep), min near -30 (delete) when healthy
<code>delete_gate_loss_coeff</code>	Current α (PI controller output)	Should stabilize once deletion rate converges

Table 2.3: Delete gate metrics.

2.3.3 Optimization Metrics

Metric	What It Measures	Expected Behavior
<code>learning_rate</code>	Current LR from linear schedule	Starts at 2e-5, decays to 0 by final step
<code>epoch</code>	Current epoch (1-indexed)	Increments at ~17,168 step intervals

Table 2.4: Optimization metrics.

2.4 Step-by-Step Analysis Workflow

2.4.1 Phase 1: Steps 0–1,000 (Pre-Regularizer)

The model is learning the classification task with no deletion pressure.

What to check:

- `cross_entropy_loss` should be dropping from ~1.1 toward ~0.7.
- `accuracy` should be rising from ~33% (random baseline for 3-class NLI).
- `delete_gate_average` at step 50 should be close to 0—this confirms the gate bias initialization (`bias=10`) is working. If near -30, all tokens are being deleted from the start (a bug).
- `new_seq_len` should be close to the full average non-pad sequence length (~25–35 tokens for SNLI). If 5–6 tokens, the gate is already aggressively deleting, indicating bias initialization failed.

Red flags:

- `accuracy` stuck at 33% after 500 steps → task loss not flowing correctly.
- `delete_gate_average` near -30 at step 50 → gate initialization not working.

2.4.2 Phase 2: Steps 1,000–5,000 (PI Controller Engages)

Deletion pressure kicks in at step 1,000. The PI controller begins adjusting α to push the deletion rate toward the 30% target.

What to check:

- `percent_non_pad_deleted_tokens` should start rising from near 0% toward 30%.
- `delete_gate_loss_coeff` (α) should start increasing from 0.01.
- `new_seq_len` will drop as deletion starts—expect a temporary spike or dip as the controller stabilizes. Some oscillation is normal.
- `delete_gate_std` should be increasing—the gate is learning to discriminate.

Red flags:

- `percent_non_pad_deleted_tokens` jumps to 70–80% → controller gain too high; reduce `controller_p`.
- `delete_gate_std` stays near 0 → gate not learning to discriminate.

2.4.3 Phase 3: Steps 5,000–17,168 (End of Epoch 1)

The gate and task head should be co-adapting.

What to check:

- `percent_non_pad_deleted_tokens` should be trending toward 30%, possibly still noisy.
- `cross_entropy_loss` should still be decreasing.
- `accuracy` should be in the 75–85% range by end of epoch 1 for a healthy run.
- `new_seq_len` should be stabilizing around `avg_seq_len` × 0.70.

2.4.4 Phase 4: Steps 17,168–51,500 (Epochs 2–3)

Convergence phase. Both task performance and deletion rate should be settling.

What to check:

- `eval/accuracy` is the key metric—compare against BERT baseline.
- `percent_non_pad_deleted_tokens` should be within $\pm 5\%$ of the 30% target and no longer trending.
- `cross_entropy_loss` curve should be flattening.
- `learning_rate` should still be non-zero—if it hits 0 before step 51,500, the scheduler was set for fewer steps (fix: use `-max-steps -1`).

2.5 Parameter Adjustments by Symptom

2.5.1 Deletion Rate Never Reaches Target

2.5.2 Gate Not Discriminating (Low `delete_gate_std`)

The gate outputs similar values for all tokens—it isn't learning what to delete.

Symptom	Likely Cause	Adjustment
Rate plateaus at 10–20%	Controller too slow	Increase <code>controller_p</code> from 0.01 to 0.05
Rate oscillates wildly	Controller overshooting	Decrease <code>controller_p</code> to 0.005
Rate converges slowly	Integral term too weak	Increase <code>controller_i</code> from 0.00001 to 0.0001
Rate instantly hits 80%+ at step 1000	Gate collapse	Increase <code>regularizer_delay</code> to 2000–3000

Table 2.5: Deletion rate troubleshooting.

Adjustment	Effect
Increase <code>-deletion_loss_weight</code>	Stronger deletion pressure from the start
Increase <code>-regularizer_delay</code>	More time to learn task before deletion pressure
Try <code>-use_gumbel_noise</code>	Adds exploration noise to prevent gate from getting stuck

Table 2.6: Adjustments for low gate discrimination.

2.5.3 Task Performance Worse Than BERT Baseline

2.5.4 Learning Rate Issues

2.6 Success Criteria

A training run can be considered successful when **all** of the following are met:

2.6.1 Task Performance

- `eval/accuracy` $\geq$ **89%** on SNLI validation set.
- MrBERT `eval/accuracy` is within **1–2%** of the BERT baseline.
- `eval/accuracy` has **plateaued**—the last 5,000 steps show less than 0.5% improvement.

2.6.2 Delete Gate Convergence

- `percent_non_pad_deleted_tokens` is within $\pm 3\%$ of the target deletion rate.
- `percent_non_pad_deleted_tokens` is **stable**—variance over the last 2,000 steps is less than $\pm 5\%$.
- `delete_gate_std` is non-trivial (> 1.0)—the gate is making meaningful distinctions.

2.6.3 Training Stability

- `cross_entropy_loss` has **plateaued**.
- `delete_gate_loss_coeff` (α) has **stabilized**.
- `new_seq_len` is **stable**—consistent effective sequence length.

Symptom	Likely Cause	Adjustment
Accuracy lags behind throughout	Deletion too aggressive	Reduce <code>target_deletion_rate</code> (e.g. 0.2)
Accuracy starts equal then diverges	Gate deleting informative tokens	Increase <code>regularizer_delay</code>
Eval accuracy drops after epoch 1	Overfitting	Reduce epochs or add weight decay

Table 2.7: Task performance troubleshooting.

Symptom	Likely Cause	Adjustment
Loss stops improving mid-run	LR decayed too early	Use <code>-max-steps -1</code>
Loss unstable, large spikes	LR too high	Reduce to <code>1e-5</code>

Table 2.8: Learning rate troubleshooting.

2.6.4 Comparison Against Baseline

Run a BERT baseline with identical hyperparameters (`-model-type BERT`) and compare:

- `eval/accuracy`: MrBERT should be within 2% of BERT.
- Effective compute per forward pass: `new_seq_len / original_seq_len` should be significantly < 1.0 .

2.6.5 Summary Table

Metric	Success Threshold
<code>eval/accuracy</code>	$\geq 89\%$
Gap vs BERT baseline	$\leq 2\%$
<code>percent_non_pad_deleted_tokens</code>	within $\pm 3\%$ of target
<code>delete_gate_std</code>	> 1.0
<code>cross_entropy_loss</code> slope	< 0.001 per 1000 steps
<code>new_seq_len</code> stability	$\pm < 1$ token over last 2000 steps

Table 2.9: Success criteria summary.

2.6.6 When to Stop Early

Stop the run and adjust parameters if by the end of epoch 1 ($\sim 17,168$ steps):

- `accuracy` $< 60\%$ (task not learning).
- `percent_non_pad_deleted_tokens` $> 70\%$ consistently (gate has collapsed).
- `delete_gate_std` < 0.1 (gate not discriminating).
- `new_seq_len` < 5 tokens (almost all content deleted).

Chapter 3

Training Run Plan

A structured plan for training runs with different parameter variations, including the intuition behind each run and the expected metric outcomes. Runs are ordered from foundational baselines to ablation studies.

For metric interpretation and success criteria, see Chapter 2.

3.1 Overview

Run	Model	Purpose	Key Variation
1	BERT	Baseline reference	No delete gate
2	MrBERT	Primary model	Default settings, 30% deletion
3	MrBERT	Compression vs accuracy tradeoff	50% deletion target
4	MrBERT	Conservative compression	20% deletion target
5	MrBERT	PI controller ablation	Fixed α , no dynamic adjustment
6	MrBERT	Gate placement ablation	Gate at layer 1 instead of 3
7	MrBERT	Gate placement ablation	Gate at layer 6 instead of 3
8	MrBERT	Learned vs random deletion	Random gate, 30% deletion

Table 3.1: Training run plan overview.

3.2 Run 1: BERT Baseline

```
modal run --detach train_modal.py \  
  --model-type BERT \  
  --max-steps -1 \  
  --num-epochs 3 \  
  --mode training-and-eval
```

Intuition: This is the reference point for all other runs. Standard BERT fine-tuned on SNLI with no deletion. Every subsequent MrBERT run will be measured against this. Without this baseline, there is no way to know whether MrBERT’s accuracy drop (if any) is an acceptable tradeoff for the efficiency gain.

Metric	Expected Value
eval/accuracy	~90–91%
cross_entropy_loss	Smooth decay to ~0.25–0.30 by epoch 3
accuracy (train)	~91–92%
Delete gate metrics	N/A—no gate

Table 3.2: Run 1 expected metrics.

What to look for: This establishes the accuracy ceiling. Any MrBERT run that matches within $\sim 1\text{--}2\%$ is a successful compression. If BERT baseline underperforms ($< 88\%$), it likely indicates a data or training setup issue that would affect all subsequent runs.

3.3 Run 2: MrBERT Standard

```
modal run --detach train_modal.py \
  --model-type MrBERT \
  --max-steps -1 \
  --num-epochs 3 \
  --target-deletion-rate 0.3 \
  --mode training-and-eval
```

Intuition: This is the primary MrBERT run. The 30% deletion target is chosen as a moderate tradeoff: enough compression to reduce compute meaningfully ($\sim 49\%$ reduction in attention compute for layers 4–11, since attention scales as $O(n^2)$), while being conservative enough that the model retains most of the informative tokens. The PI controller is enabled to dynamically stabilize the deletion rate at exactly 30%.

Metric	Expected Value
eval/accuracy	88–90%
percent_non_pad_deleted_tokens	Converges to 27–33% by epoch 2
delete_gate_std	> 1.0
new_seq_len	$\sim 19\text{--}22$ tokens
delete_gate_loss_coeff	Stabilizes after initial oscillation
cross_entropy_loss	Slightly higher than baseline

Table 3.3: Run 2 expected metrics.

What to look for: Compare eval/accuracy directly against Run 1. The gap is the cost of 30% compression. A rising `delete_gate_std` indicates the gate is learning which tokens to delete, not just uniformly suppressing everything.

3.4 Run 3: MrBERT Higher Deletion Rate

```
modal run --detach train_modal.py \
  --model-type MrBERT \
  --max-steps -1 \
  --num-epochs 3 \
  --target-deletion-rate 0.5 \
  --mode training-and-eval
```

Intuition: Deleting 50% of tokens is a much more aggressive compression. The model must learn to identify the most informative half of each sequence. In NLI, many tokens are redundant—determiners, prepositions, repeated words—so there is reason to believe 50% deletion is viable.

What to look for: If accuracy drops more than 3–4% vs. baseline, 50% deletion is too aggressive. Watch for gate collapse early in training—with a 50% target the PI controller applies stronger pressure, which can destabilize the gate in the first epoch.

Metric	Expected Value
eval/accuracy	85–88%
percent_non_pad_deleted_tokens	Converges to ~ 47 –53%
new_seq_len	~ 12 –15 tokens
delete_gate_std	Higher than Run 2
cross_entropy_loss	Higher than Run 2 throughout

Table 3.4: Run 3 expected metrics.

3.5 Run 4: MrBERT Lower Deletion Rate

```
modal run --detach train_modal.py \
  --model-type MrBERT \
  --max-steps -1 \
  --num-epochs 3 \
  --target-deletion-rate 0.2 \
  --mode training-and-eval
```

Intuition: Deleting only 20% of tokens is a conservative setting. The expectation is that accuracy should be very close to the BERT baseline ($\sim 36\%$ reduction in attention compute for layers 4–11). This is also a sanity check: if 20% deletion significantly hurts accuracy, something is fundamentally wrong with the gate-task interaction.

Metric	Expected Value
eval/accuracy	89–91%
percent_non_pad_deleted_tokens	Converges to ~ 17 –23%
new_seq_len	~ 22 –27 tokens
delete_gate_std	Lower than Run 2
cross_entropy_loss	Very close to baseline

Table 3.5: Run 4 expected metrics.

What to look for: If accuracy matches within 0.5%, the gate is truly selecting only redundant tokens. Together with Runs 2 and 3, this completes the accuracy-vs-compression tradeoff curve: 20% $\rightarrow$ 30% $\rightarrow$ 50%.

3.6 Run 5: MrBERT No PI Controller

```
modal run --detach train_modal.py \
  --model-type MrBERT \
  --max-steps -1 \
  --num-epochs 3 \
  --mode training-and-eval \
  --no-use-pi-controller
```

Intuition: Without the PI controller, α stays fixed at `-deletion_loss_weight` (default 0.01) for the entire run. This directly tests the value of the PI controller. Two outcomes are possible:

1. The deletion rate lands near 30% anyway $\rightarrow$ the PI controller adds little value.

2. The deletion rate drifts far from 30% → the PI controller is essential.

The intuition from the MrT5 paper is that a fixed α is fragile: even small changes in learning dynamics can push the gate to collapse or become inactive.

Metric	Expected Value
eval/accuracy	Similar to Run 2, or worse if gate drifts
percent_non_pad_deleted_tokens	Unpredictable—likely drifts from 30%
delete_gate_loss_coeff	Flat line at 0.01 throughout
cross_entropy_loss	May be lower if less deletion pressure

Table 3.6: Run 5 expected metrics.

3.7 Run 6: MrBERT Earlier Gate Layer

```
modal run --detach train_modal.py \
  --model-type MrBERT \
  --max-steps -1 \
  --num-epochs 3 \
  --target-deletion-rate 0.3 \
  --mode training-and-eval
```

Pass `-delete_gate_layer 1` via extra args, or change `delete_gate_layer` default in `configuration_mrbert.py` to 1.

Intuition: Moving the gate to layer 1 means deletion decisions are made almost immediately—the model sees each token’s representation after only 1 layer of context. The hypothesis: earlier deletion means more layers (2–11) benefit from the compressed sequence, improving efficiency—but accuracy may suffer because the gate has less information to work with.

Metric	Expected Value
eval/accuracy	Lower than Run 2
percent_non_pad_deleted_tokens	Converges to ~30%
delete_gate_std	May be lower than Run 2
cross_entropy_loss	Likely higher than Run 2

Table 3.7: Run 6 expected metrics.

3.8 Run 7: MrBERT Later Gate Layer

Change `delete_gate_layer` default in `configuration_mrbert.py` to 6, or pass via extra args.

Intuition: Moving the gate to layer 6 (the halfway point) means the model has built rich contextual representations before making deletion decisions. The gate should make better-informed choices. However, since only layers 7–11 (5 layers) benefit from the compressed sequence, the efficiency gain is smaller. This run tests whether gate quality (better decisions) outweighs gate placement (earlier = more efficient).

What to look for: Together with Runs 2 and 6, this completes the gate layer sweep: layer 1 → layer 3 → layer 6. If accuracy at layer 6 is not meaningfully better than layer 3, layer 3 is the better choice because it provides more efficiency gain for the same accuracy.

Metric	Expected Value
eval/accuracy	Higher than Run 2 and Run 6
percent_non_pad_deleted_tokens	Converges to $\sim 30\%$
delete_gate_std	Higher than Run 2
cross_entropy_loss	Likely lower than Run 2

Table 3.8: Run 7 expected metrics.

3.9 Run 8: MrBERT Random Gate

Change `deletion_type` to "random" in `configuration_mrbert.py`, or pass via extra args.

Intuition: The random gate deletes 30% of non-special tokens at random, with no learning. This is the most important ablation because it directly answers: **does the model actually learn which tokens to delete, or is any 30% deletion equally good?**

If MrBERT (Run 2) significantly outperforms the random gate, it proves the learned gate is adding value—it is identifying and removing the least informative tokens. If performance is similar, token selection does not matter and any compression at the target rate performs the same. Either outcome is a meaningful finding.

Metric	Expected Value
eval/accuracy	Lower than Run 2
percent_non_pad_deleted_tokens	$\sim 30\%$ (by design)
delete_gate_std	Near 0—no learned discrimination
delete_gate_average	Constant—not learned
cross_entropy_loss	Higher than Run 2

Table 3.9: Run 8 expected metrics.

What to look for: The accuracy gap between Run 2 (learned) and Run 8 (random) is the key result. A gap of 2–3%+ means the gate is learning meaningful deletions. If the gap is small ($< 1\%$), it suggests the NLI task is robust to the specific tokens deleted—a surprising but publishable result.

3.10 Summary: What Each Run Tells You

Run	Core Question Answered
1	What is the accuracy ceiling without any compression?
2	Can MrBERT match BERT accuracy with 30% deletion?
3	How much accuracy do we sacrifice for aggressive compression?
4	Is conservative compression essentially free in terms of accuracy?
5	Is dynamic α adjustment necessary for stable, targeted deletion?
6	Does early deletion (more efficiency) hurt accuracy significantly?
7	Does late deletion (better decisions) recover accuracy?
8	Is the learned gate actually learning something meaningful?

Table 3.10: Summary of research questions addressed by each run.

3.11 Recommended Run Order

1. **Run 1** and **Run 2** in parallel—establish baseline and primary model simultaneously.
2. **Run 8**—most important ablation, run early to validate the core premise.
3. **Runs 3 and 4** in parallel—complete the compression tradeoff curve.
4. **Run 5**—PI controller ablation.
5. **Runs 6 and 7** in parallel—gate layer sweep.